



الجمهورية العربية السورية

جامعة دمشق

كلية الهندسة المعلوماتية

محرك المعالجة عالي الأداء لنظام تجارة إلكتروني

High-Performance E-Commerce Backend Engine

تقرير مشروع مادة البرمجة المتوازية

تسنيم شادي الحكيم

آية عدنان عمران

مروة محمد زيواني

سارة عدنان عطية

إيمان نديم ياغي

مقدمة

لم تعد تطبيقات التجارة الإلكترونية مجرد واجهات لعرض المنتجات، بل أصبحت منصات حيوية تتعامل مع ملايين العمليات المتزامنة، وإن قدرة هذه الأنظمة على توفير تجربة مستخدم سلسة وفورية تعد عاملاً مهماً لنجاح المنتج التجاري. وهنا تبرز الفجوة بين التطبيقات التقليدية وتلك القادرة على استغلال قدرات المعالجات والخوادم الموزعة.

هذا المشروع لا يهدف فقط إلى بناء متجر إلكتروني، بل إلى تصميم وتنفيذ هيكلية برمجية تعالج بعمق تحديات الأداء باستخدام أساليب البرمجة المتوازية.

سيركز هذا التقرير على عرض الحلول التي تم التوصل إليها مع التفسير المنهجي لسبب اختيارها مع طريقة تطبيقها ونتائج أدائها، و سنقوم بتعريف مجموعة من المتطلبات الوظيفية الأساسية كحالة دراسية، تشمل إدارة المنتجات، وعربة التسوق، وعملية الدفع، والتي ستكون بمثابة الأرضية التي نختبر ونثبت عليها نماذج التوازي والتوزيع المقترحة.

المتطلبات الوظيفية المطبقة

انشاء مستخدمين

اضافة – سحب رصيد الى المحفظة

انشاء – تعديل – عرض – حذف المنتجات

اضافة كمية للمخزون

اضافة منتج للعربة وتعديل كميته وحذفه

تاكيد الطلب واجراء الدفع

تعديل حالة الطلب

امكانية الغاء الطلب اذا كان في مرحلة الانتظار

المتطلبات غير الوظيفية المطبقة

المتطلب الأول Concurrent Access & Data Integrity

في أنظمة التجارة الإلكترونية متعددة المستخدمين، قد يحاول أكثر من مستخدم تنفيذ عمليات شراء على نفس المنتج في نفس الوقت، مما يؤدي إلى ظهور مشكلات تتعلق بتزامن العمليات (Concurrency Problems). ومن أبرز هذه المشكلات مشكلة قراءة البيانات قبل تحديثها (Dirty Read / Race Condition)، حيث يتم التحقق من كمية المنتج المتوفرة قبل أن يتم تحديث الكمية فعلياً نتيجة عملية شراء أخرى.

في نظام المتجر الإلكتروني، عند قيام المستخدم بإتمام عملية الدفع، يقوم النظام أولاً بالتحقق من توفر الكمية المطلوبة من المنتج.

لكن في حالة وجود عدة مستخدمين يحاولون شراء نفس المنتج في نفس اللحظة، قد تحدث المشكلة التالية:

- المستخدم الأول يطلب شراء آخر قطعة من المنتج .
- قبل أن يتم تحديث الكمية في قاعدة البيانات، يقوم المستخدم الثاني بإرسال طلب شراء لنفس المنتج .
- النظام يقرأ الكمية القديمة لكلا المستخدمين ويعتبر أن المنتج ما يزال متوفرًا .
- يتم قبول الطلبين معًا رغم أن الكمية الحقيقية لا تسمح بذلك .

وهذا يؤدي إلى:

- بيع منتجات غير متوفرة .
- حدوث أخطاء في المخزون .
- فقدان موثوقية النظام .
- تعارض البيانات داخل قاعدة البيانات.

الحلول المطبقة لمعالجة المشكلة

1- تنفيذ عملية التحقق والشراء داخل Transaction واحدة

تم استخدام مفهوم **Database Transaction** بحيث تصبح جميع خطوات عملية الشراء عملية واحدة مترابطة .

آلية العمل:

تشمل العملية الخطوات التالية داخل Transaction واحدة:

1. التحقق من توفر الكمية .
2. خصم الكمية من المخزون .
3. إنشاء سجل الطلب .
4. تحديث حالة الدفع .
5. حفظ جميع التغييرات .

وفي حال فشل أي خطوة من الخطوات السابقة:

- يتم إلغاء جميع التعديلات السابقة تلقائيًا (Rollback)
- تعود قاعدة البيانات إلى حالتها الأصلية قبل بدء العملية.

أما إذا نجحت جميع الخطوات: يتم تثبيت التعديلات.

أهمية هذا الحل:

هذا الأسلوب يحقق خاصية Atomicity من خصائص ACID في قواعد البيانات، والتي تعني أن العملية إما تنجح بالكامل، أو تفشل بالكامل دون ترك بيانات غير مكتملة.

2- استخدام SELECT FOR UPDATE

عند قيام مستخدم بقراءة بيانات منتج باستخدام FOR UPDATE يتم قفل الصف (Row Lock) الخاص بالمنتج داخل قاعدة البيانات واي مستخدم آخر يحاول تعديل نفس المنتج يجب أن ينتظر حتى انتهاء العملية الحالية، ويحقق:

- منع قراءة بيانات قديمة قبل تحديثها .
- ضمان تنفيذ العمليات بالتسلسل الصحيح .
- منع تضارب تحديثات المخزون .
- الحفاظ على دقة بيانات المنتجات.

The screenshot shows the Django administration interface. The top navigation bar includes 'Django administration' and user information 'WELCOME, EMAN. VIEW SITE / CHANGE PASSWORD / LOG OUT'. The breadcrumb trail is 'Home > Store > Products > iPhone'. The left sidebar contains a search bar and a list of menu items: 'AUTHENTICATION AND AUTHORIZATION' (Groups, Users), 'PERIODIC TASKS' (Clocked, Crontabs, Intervals, Periodic tasks, Solar events), and 'STORE' (Banks, Cart products). The main content area is titled 'Change product' and shows the 'iPhone' product form. The form fields are: 'Name' (iPhone), 'Price' (100.00), and 'Quantity' (1). At the bottom of the form are buttons: 'SAVE', 'Save and add another', 'Save and continue editing', and 'Delete'.

The screenshot shows a user interface for a shopping cart. It has a dark background with white text. The 'Cart:' section shows 'Aya Cart' with a dropdown arrow, a pencil icon, a plus icon, and an eye icon. Below it, the 'Product:' section shows 'iPhone' with a dropdown arrow, a pencil icon, a plus icon, and an eye icon. The 'Quantity:' section shows the number '1' in a text input field.

Cart:

marwa Cart ▾



Product:

iPhone ▾



Quantity:

1

```
(venv) D:\ecommerce\PP-Ecommerce-Project>python test_concurrency.py
```

marwa

Status: 400

Response: {"message":"Cart is empty"}

Aya

Status: 200

Response: {"message":"Payment successful","order_id":1}

```
(venv) D:\ecommerce\PP-Ecommerce-Project>
```

النتيجة النهائية

تم ضمان سلامة البيانات ومنع تضارب العمليات المتزامنة، وموثوقية عمليات الدفع والشراء داخل النظام، كما ساهم هذا الحل في جعل النظام قادرا على التعامل مع عدد كبير من المستخدمين في نفس الوقت دون حدوث أخطاء في بيانات المنتجات أو الطلبات.

المتطلب الثاني Resource Management & Capacity

عملية إدارة الموارد تُعنى بالاستهلاك المناسب لموارد الحاسوب من أجل تنفيذ عدد من العمليات على التوازي دون حدوث تضارب ودون حدوث بطء بالتنفيذ

طرق التنفيذ:

هناك عدة طرق ممكنة لإدارة هذه الموارد وفعليا لا يتم اختيار أحد هذه الطرق تفضيلا عن الأخرى وإنما يتم التكامل بينها جميعا لتعطي النتيجة النهائية, حيث يتم اختيار ما يتم تطبيقه من هذه الطرق بحسب نتيجة اختبارات لاستهلاك الموارد وليس بشكل اعتباطي, ومن هذه الطرق: thread pooling, connection pooling, queue, async queues, batch processing

في هذا الطلب تم تطبيق مفاهيم ال connection pooling, thread pooling and queue

بينما async queues and batch processing فقد تم تطبيقهم بالطلب الثالث والرابع

سبب اختيار تطبيق هذه المفاهيم:

بحسب نتائج الاختبار, يتضح أن موارد الجهاز يتم استهلاكها بشكل كبير ويتم إنشاء مئات threads ومع ذلك يفشل النظام في تلبية جميع الطلبات, لذلك تطبيق إدارة ال threads وإدارة الاتصال مع قاعدة البيانات تساعد كثيرا على الاستفادة الصحيحة من الموارد في هذه الحالة

شرح المفاهيم المطبقة:

Thread pool -1

ما تم تطبيقه هو شيء مشابه لمفاهيم ال thread pool عن طريق استخدام سيرفر waitress الذي يقدم خدمة multithreading بشكل داخلي ويحتوي على queue لتنظيم ال requests عندما تمتلئ ال threads عند دراسة نظام ال backend تبين أن معظم العمليات هي I/O bound وأغلبها تتعلق بالوصول لقاعدة المعطيات, لهذا السبب تم اعتماد تحديد عدد ال:

- Threads عن طريق المعادلة: $threads = cpu_cores * 2$
أما الاستهلاك من ناحية ال cpu فهو قليل لذلك لا حاجة ل workers إضافية
- تحديد عدد ال connection ، وهو عدد الاتصالات المسموح بها بشكل متزامن مع السيرفر في نفس اللحظة, سواء كانت Requests قيد المعالجة أو Requests تنتظر ضمن ال queue.
تم تحديده باستخدام المعادلة: $connections = threads * 50$
وذلك للسماح باستقبال عدد كبير من الاتصالات دون رفضها مباشرة عند امتلاء ال threads, حيث يتم وضع الطلبات الزائدة ضمن ال queue إلى حين توفر thread فارغ لمعالجتها.
اختيار قيمة أكبر من عدد ال threads مهم في الأنظمة ال I/O bound لأن أغلب ال threads تقضي وقتها بانتظار

عمليات قاعدة البيانات ولا تكون مستهلكة للـ CPU بشكل فعلي, وبالتالي يمكن استقبال عدد اتصالات أكبر من عدد الـ threads نفسه دون التأثير الكبير على المعالج.

connection pooling -2

قمنا بتطبيق المفهوم عن طريق تحديد المدة التي يبقى فيها الاتصال مفتوحاً مع قاعدة البيانات, حيث عملية إغلاق الاتصال وإنشاءه مرة ديدة لكل request مع الـ database يستغرق وقت وموارد أيضاً, وبما أننا نحاول استهلاك الموارد بشكل متوازن لتحسين الإنتاجية, لجأنا لهذا الحل كي لا يستهلك الاتصال بقاعدة البيانات بحد ذاته موارد ووقت

الاختبارات:

تم الاختبار على الـ create_order api (اسمه في الـ views هو cart_confirm_payment)

وتم اختبار 300 طلب بمعدل 10 طلبات كل ثانية

قبل التحسين:

لا يوجد connection pooling ولا يتم إدارة الـ threads

 LOCUST

Host

http://127.0.0.1:8000/

Status

RUNNING

Users

300

RPS

45.13

Failures

4%

EDIT

LOADING

RESET



STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS

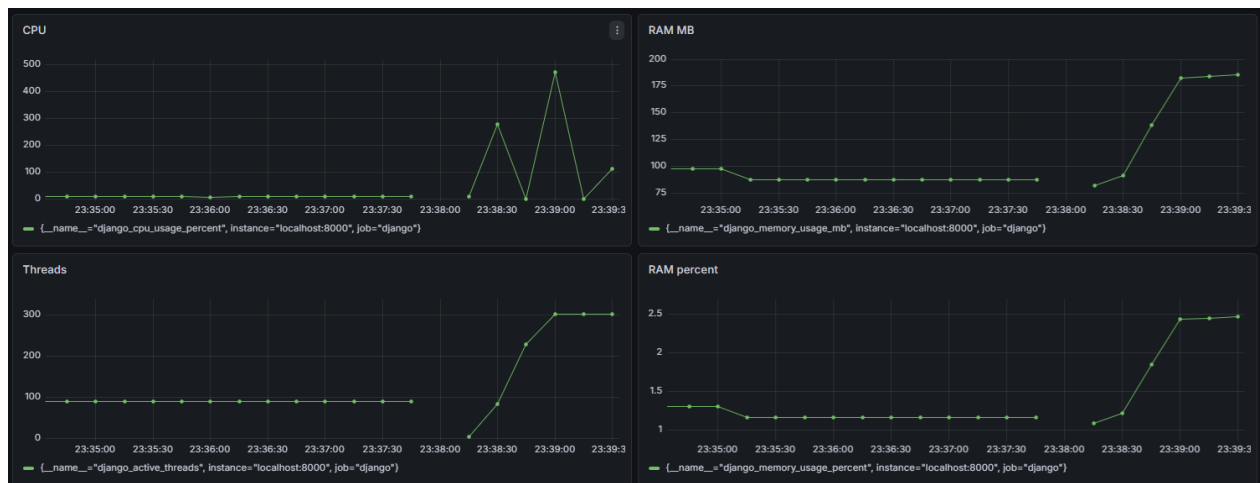
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	//api/cart/confirm/1	9	0	200	6800	6800	1785.81	56	6752	27	0.2	0
POST	//api/cart/confirm/10	1	1	11256.58	11000	11000	11256.58	11257	11257	228158	0.1	0.1
POST	//api/cart/confirm/100	1	1	13209.61	13000	13000	13209.61	13210	13210	228183	0.1	0.1
POST	//api/cart/confirm/102	25	0	140	6600	7700	1180.23	12	7657	27	0.4	0
POST	//api/cart/confirm/107	8	0	130	7100	7100	1919.36	11	7065	27	0.1	0
POST	//api/cart/confirm/109	1	1	16037	16000	16000	16037	16037	16037	228183	0.1	0.1
POST	//api/cart/confirm/11	10	0	140	8000	8000	1494.37	18	8035	27	0.1	0
POST	//api/cart/confirm/111	21	0	240	6500	8300	1507.79	12	8276	27	0.4	0
POST	//api/cart/confirm/113	16	1	110	8600	8600	1303.75	11	8591	14286.75	0.2	0
POST	//api/cart/confirm/115	25	0	180	6400	7300	1245.95	9	7280	27	0.4	0
POST	//api/cart/confirm/12	1	1	18828.95	19000	19000	18828.95	18829	18829	228158	0	0
POST	//api/cart/confirm/120	38	0	120	7100	8200	1251.02	9	8180	27	0.5	0
POST	//api/cart/confirm/122	9	1	1400	15000	15000	3370.88	17	15025	25377.67	0.3	0.1
POST	//api/cart/confirm/123	1	1	16972.13	17000	17000	16972.13	16972	16972	228183	0.1	0.1

STATISTICS	CHARTS	FAILURES	EXCEPTIONS	CURRENT RATIO	DOWNLOAD DATA	LOGS
# Failures	Method	Name	Message	First Seen	Last Seen	
1	POST	//api/cart/confirm/10	HTTPError(500 Server Error: Internal Server Error for url: //api/cart/confirm/10)	5/18/2026, 10:39:00 PM	5/18/2026, 10:39:00 PM	
1	POST	//api/cart/confirm/100	HTTPError(500 Server Error: Internal Server Error for url: //api/cart/confirm/100)	5/18/2026, 10:39:02 PM	5/18/2026, 10:39:02 PM	
1	POST	//api/cart/confirm/109	HTTPError(500 Server Error: Internal Server Error for url: //api/cart/confirm/109)	5/18/2026, 10:39:06 PM	5/18/2026, 10:39:06 PM	
1	POST	//api/cart/confirm/113	HTTPError(500 Server Error: Internal Server Error for url: //api/cart/confirm/113)	5/18/2026, 10:38:52 PM	5/18/2026, 10:38:52 PM	
1	POST	//api/cart/confirm/12	HTTPError(500 Server Error: Internal Server Error for url: //api/cart/confirm/12)	5/18/2026, 10:39:12 PM	5/18/2026, 10:39:12 PM	



الاختبار يوضح مايلي:

الوقت الذي يستغرقه لمعالجة الطلب يصل في الكثير من الحالات لـ 20 ثانية، وحصل فشل في عدة طلبات بسبب خطأ داخلي بالسيرفر نتيجة الضغط إضافة إلى أن معدل عدد الطلبات في الثانية RPS : تقريبا 45



أما بالنسبة لاستهلاك الموارد فتوضح المخططات أن الاستهلاك عالي جدا حيث يستهلك تقريبا 90% من ال CPU وقام بإنشاء ما يقارب 300 thread ورغم ذلك كان هناك بعض الطلبات التي تفشل

بعد إجراء التحسينات:

LOCUST												Host http://127.0.0.1:8000/	Status STOPPED	RPS 136.87	Failures 0%	NEW	RESET	
STATISTICS																		
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s						
POST	//api/cart/confirm/1	65	0	91	220	290	95.63	9	288	27	1.3	0						
POST	//api/cart/confirm/100	65	0	92	200	260	96.36	9	256	27	1.2	0						
POST	//api/cart/confirm/102	30	0	130	290	350	135.02	9	345	27	0.7	0						
POST	//api/cart/confirm/103	28	0	120	220	270	120.15	37	270	27	0.6	0						
POST	//api/cart/confirm/104	23	0	140	330	330	137.99	14	334	27	0.6	0						
POST	//api/cart/confirm/105	62	0	160	320	380	162.29	11	376	27	1.7	0						
POST	//api/cart/confirm/106	58	0	180	290	410	183.2	15	407	27	1.8	0						
POST	//api/cart/confirm/107	32	0	90	230	290	110.02	10	286	27	0.6	0						
POST	//api/cart/confirm/109	27	0	140	210	250	121.2	15	248	27	0.6	0						
POST	//api/cart/confirm/11	61	0	97	240	350	103.91	11	350	27	1.4	0						
POST	//api/cart/confirm/110	22	0	150	240	350	151.85	37	345	27	0.6	0						
POST	//api/cart/confirm/111	26	0	140	220	220	136.45	10	217	27	0.6	0						
POST	//api/cart/confirm/112	29	0	110	280	310	114.27	12	309	27	0.6	0						
POST	//api/cart/confirm/113	56	0	130	280	360	122.46	13	364	27	1.2	0						

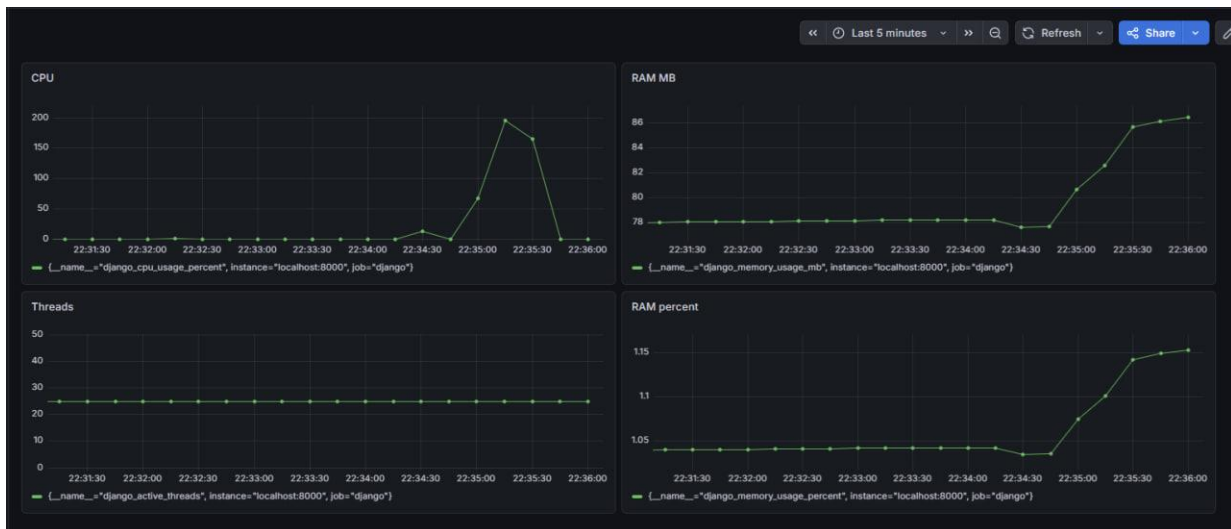


```
(PP-Ecommerce-Project) PS C:\Users\Tsunami\Desktop\PP-Ecommerce-Project> python run_server.py

=====
CPU cores: 12
Threads: 24
Connection limit: 1200
=====

INFO:waitress:erving on http://127.0.0.1:8000
WARNING:waitress.queue:Task queue depth is 1
WARNING:waitress.queue:Task queue depth is 1
WARNING:waitress.queue:Task queue depth is 1
WARNING:waitress.queue:Task queue depth is 1
WARNING:waitress.queue:Task queue depth is 1
WARNING:waitress.queue:Task queue depth is 1
WARNING:waitress.queue:Task queue depth is 1
WARNING:waitress.queue:Task queue depth is 1
WARNING:waitress.queue:Task queue depth is 1
WARNING:waitress.queue:Task queue depth is 1
WARNING:waitress.queue:Task queue depth is 1
WARNING:waitress.queue:Task queue depth is 1
```

نلاحظ تحسن كبير بزمان الاستجابة (وسطيا ms400) ولا يوجد أي أخطاء أو فشل اتصال، والصورة الأخيرة توضح أن ال queue لا تحتوي إلا على request واحد تقريبا دائما مما يعني أن عدد الطلبات التي تنتظر لتتم معالجتها قليل



نلاحظ أن استهلاك الموارد أصبح متوازن وطبيعي، ورغم أن الاستهلاك هنا أقل إلى أنه بسبب الإدارة الصحيحة وتنظيم ال thread كانت الإنتاجية أفضل

المتطلب الثالث Asynchronous Queues

في الأنظمة الحديثة، تُعدّ مسألة تحسين الأداء وتقليل زمن الاستجابة من أهم التحديات، خاصة عند التعامل مع عمليات ثقيلة مثل توليد الملفات أو إرسال البريد الإلكتروني. في هذا المشروع، ظهرت الحاجة إلى معالجة هذه المشكلة من خلال الانتقال من التنفيذ المتزامن (Synchronous) إلى التنفيذ غير المتزامن.

المشكلة قبل تطبيق الحل:

في النسخة الأولى من النظام، كانت عملية تأكيد الطلب (Order Confirmation) تتضمن تنفيذ عدة مهام داخل نفس الطلب (HTTP Request)، وهي:

- التحقق من الدفع
- إنشاء الطلب

وعند نجاح الطلب يتم :

- توليد فاتورة بصيغة PDF
- إرسال الفاتورة عبر البريد الإلكتروني

وكانت هذه العمليات تنفذ بشكل متسلسل ومتزامن، مما أدى إلى:

- زيادة زمن الاستجابة بشكل ملحوظ، حيث كان المستخدم ينتظر حتى انتهاء جميع العمليات.
- تعطيل (Blocking) الخيط التنفيذي (Thread) الخاص بالطلب، مما يقلل من قدرة النظام على معالجة طلبات أخرى.
- قابلية عالية للفشل، إذ إن فشل أي جزء (مثل إرسال البريد) يؤدي إلى فشل العملية كاملة.
- تجربة مستخدم غير جيدة نتيجة التأخير.

الهدف من الحل:

- فصل العمليات الثقيلة عن الطلب الأساسي
- تنفيذ هذه العمليات في الخلفية
- تقليل زمن الاستجابة إلى الحد الأدنى
- تحسين اعتمادية النظام (Reliability)

الحل المعتمد:

تم اعتماد البنية التالية:

- استخدام Celery كإطار لتنفيذ المهام غير المتزامنة

- استخدام Redis كوسيط لتخزين المهام

آلية العمل:

- 1- يقوم المستخدم بإرسال طلب تأكيد الشراء عبر الواجهة (API Request).
 - 2- عند استقبال الطلب، يقوم النظام أولاً بمحاكاة مرحلة معالجة أولية (Processing Delay) لتمثيل الزمن الذي تستغرقه الأنظمة عادة في التحقق من البيانات، وذلك باستخدام تأخير زمني بسيط.
 - 3- بعد ذلك، يتم تنفيذ محاكاة لعملية الدفع (Payment Simulation)، حيث:
 - يتم إدخال تأخير إضافي يمثل زمن التواصل مع النظام البنكي.
 - يتم تحديد نتيجة الدفع (نجاح أو فشل) بشكل عشوائي لمحاكاة سلوك الأنظمة الواقعية.
 - 4- في حال نجاح عملية الدفع (Payment Success):
 - يتم إنشاء الطلب (Order) وتحديث حالة النظام.
 - عند نجاح الطلب يتم إرجاع الاستجابة للمستخدم مباشرة دون انتظار العمليات اللاحقة.
 - 5- بالتوازي، يقوم النظام بإرسال مهمة (Task) إلى Celery عبر Redis Broker، دون إيقاف أو تأخير استجابة المستخدم.
 - 6- يقوم Celery Worker بالتقاط المهمة وتنفيذها في الخلفية، وتشمل:
 - محاكاة معالجة طويلة نسبياً (Heavy Processing Delay)
 - توليد ملف الفاتورة (PDF Invoice)
 - إرسال بريد إلكتروني للمستخدم مرفقاً بالفاتورة
 - 7- في حال حدوث أي خطأ أثناء تنفيذ المهمة:
 - يتم إعادة المحاولة (Retry) بشكل تلقائي وفق سياسة محددة.
 - وفي حال فشل جميع المحاولات، يتم اعتبار المهمة فاشلة ومعالجتها لاحقاً.
- عند تصميم نظام معالجة مهام غير متزامنة (Asynchronous Processing)، كان لا بد من اختيار وسيط رسائل (Message Broker) مناسب لإدارة تنفيذ المهام الخلفية عبر Celery. وقد تم دراسة ثلاث خيارات رئيسية:

العيوب	المميزات	آلية العمل	
لا يدعم بشكل افتراضي: Dead Letter Queue و Advanced Routing	سرعة عالية جداً (Low Latency) سهل الإعداد (Lightweight) مناسب للأنظمة التي تحتاج أداء سريع (مثل إرسال الإيميلات)	يقوم Celery بوضع الرسائل (Tasks) ضمن Redis Queue	Redis

يقوم Worker بسحب الرسائل وتنفيذها يتم تخزين الرسائل في الذاكرة (RAM) مما يجعل الأداء سريع جداً		في حال انطفاء السيرفر ممكن فقدان بعض الرسائل (إذا لم يتم تفعيل persistence)
RabbitMQ الرسائل تُرسل إلى Exchange يتم توجيهها إلى Queue بناءً على Routing Rules Worker يستهلك الرسائل من Queue	دعم قوي جداً لـ Dead Letter Queues (DLQ) و Retry Policies و Message Acknowledgement وضمان أعلى لوصول الرسائل (Reliability) و مناسب للأنظمة الكبيرة (Enterprise Systems)	أعقد في الإعداد أبسط نسبياً من Redis بسبب الكتابة على Disk وإدارة الرسائل بشكل أكثر أماناً
Memory Broker هو وسيط مؤقت يعتمد على الذاكرة فقط داخل نفس العملية.	سريع جداً (لأنه لا يوجد Network) لا يحتاج أي إعداد خارجي	لا يعمل بين أكثر من Process لا يصلح للإنتاج الرسائل تضيق عند إيقاف البرنامج

التقنية المختارة:

تم اختيار Redis لأن النظام الحالي لا يحتاج تعقيد RabbitMQ فالمطلوب إرسال إيميلات وإنشاء PDF، كما ان الأداء السريع أهم من التعقيد، ولسهولة الإعداد والتشغيل.

قبل تطبيق الحل كان المستخدم ينتظر عند الدفع على الأقل 10 ثواني وفي الحياة الواقعية والردود البنكية قد تستغرق الموافقة أكثر

اما عند تطبيق الحل ينتظر المستخدم ثانيتين فقط لرد ال payment gateways وعند نجاح الطلب ينتظر المستخدم عملية انشاء ال pdf وارسال ال email والتي قد تستغرق الى 4 ثواني عدا حالات الفشل اما بعد تطبيق الحال فلا ينتظر ابدا.

معالجة الأخطاء:

تمت إضافة آلية لمعالجة الأخطاء داخل مهام Celery بهدف ضمان استمرارية النظام وعدم فقدان العمليات في حال حدوث أعطال غير متوقعة.

تعتمد هذه الآلية على إحاطة منطق المهمة ببنية try/except، بحيث يتم التقاط أي استثناء (Exception) قد يحدث أثناء تنفيذ المهمة، سواء كان ناتجاً عن:

- فشل في الاتصال (مثل خدمة البريد الإلكتروني)
- خطأ في توليد ملف الفاتورة
- مشاكل في الوصول إلى قاعدة البيانات

عند حدوث أي خطأ، لا يتم إنهاء المهمة مباشرة، وإنما يتم تفعيل آلية إعادة المحاولة (Retry) الخاصة بـ Celery، وذلك عبر استدعاء إعادة جدولة نفس المهمة للتنفيذ مرة أخرى لاحقاً.

وهذا يمنع فشل المهمة بشكل نهائي عند أول خطأ، ويزيد من موثوقية النظام (Reliability).

آلية إعادة المحاولة:

تم ضبط إعدادات إعادة المحاولة في المهمة كما يلي:

- الحد الأقصى لعدد المحاولات: 3 مرات
- الفاصل الزمني بين كل محاولة: 5 ثواني

أي أن النظام يعمل وفق السلوك التالي:

- 1- عند حدوث خطأ أثناء تنفيذ المهمة يتم إيقاف التنفيذ الحالي مؤقتاً.
- 2- يقوم Celery بإعادة إرسال المهمة إلى الـ Queue لتتم معالجتها مرة أخرى بعد 5 ثواني.
- 3- يتم تكرار هذه العملية حتى 3 محاولات كحد أقصى.
- 4- في حال استمرار الفشل بعد جميع المحاولات يتم اعتبار المهمة فاشلة بشكل نهائي.

المتطلب الرابع Batch Processing

في الطريقة التقليدية لعملية الجرد وإرسال التقارير تتم معالجة كل عملية بيع أو سجل بشكل منفصل وضمن مسار تنفيذ واحد فقط.

حيث يقوم النظام بقراءة جميع سجلات المبيعات واحداً تلو الآخر، ثم حساب الإجماليات، وتحديث البيانات، وإنشاء التقارير بشكل متتابع. وبعد الانتهاء من جميع العمليات يتم إرسال التقرير النهائي.

ومع زيادة حجم البيانات تصبح هذه الطريقة بطيئة وتستهلك موارد كبيرة، كما أن أي خطأ أو تأخير قد يؤثر على عملية الجرد بالكامل، لذلك فهي غير مناسبة لمعالجة البيانات الضخمة.

لحل مشكلة معالجة عدد كبير من الطلبات دفعة واحدة، تم استخدام مفهوم الـ Batch Processing، بحيث يتم تقسيم البيانات إلى أجزاء صغيرة (Chunks) بدلاً من معالجة جميع الطلبات دفعة واحدة.

بداية الحل Fixed Chunking:

في البداية تم تجربة أسلوب Fixed Chunking حيث يتم تقسيم البيانات إلى batches ثابتة الحجم.
مثال:

`chunk_size = 500`

طريقة العمل:

1. جلب أول 500 Order
2. حساب المبيعات الخاصة بهم
3. الانتقال إلى الـ 500 التالية
4. تكرار العملية حتى انتهاء جميع الطلبات

مشاكل Fixed Chunking:

رغم أن هذا الحل أفضل من معالجة جميع البيانات دفعة واحدة، لكنه كان يحتوي على مشكلة مهمة:

1. حجم الـ chunk ثابت دائماً
2. لا يتغير حسب الضغط أو سرعة التنفيذ
3. قد يكون كبير جداً فيسبب ضغط على النظام أو صغير جداً فيسبب عدد كبير من الـ queries وبالتالي بطء غير ضروري .

أي أن النظام لم يكن يتكيف مع حالة السيرفر أو حجم البيانات.

```
0-ecf20005d059] Received
[2026-05-18 17:53:00,050: WARNING/MainProcess] =====
[2026-05-18 17:53:00,051: WARNING/MainProcess] Current chunk size: 100
[2026-05-18 17:53:00,052: WARNING/MainProcess] Execution time: 0.0099 sec
[2026-05-18 17:53:00,053: WARNING/MainProcess] Orders processed: 100
[2026-05-18 17:53:00,054: WARNING/MainProcess] =====
=====
[2026-05-18 17:53:00,064: WARNING/MainProcess] =====
[2026-05-18 17:53:00,077: WARNING/MainProcess] Current chunk size: 100
[2026-05-18 17:53:00,081: WARNING/MainProcess] Execution time: 0.0244 sec
[2026-05-18 17:53:00,083: WARNING/MainProcess] Orders processed: 100
[2026-05-18 17:53:00,083: WARNING/MainProcess] =====
=====
[2026-05-18 17:53:00,089: WARNING/MainProcess] =====
[2026-05-18 17:53:00,089: WARNING/MainProcess] Current chunk size: 100
[2026-05-18 17:53:00,090: WARNING/MainProcess] Execution time: 0.0489 sec
[2026-05-18 17:53:00,091: WARNING/MainProcess] Orders processed: 100
[2026-05-18 17:53:00,091: WARNING/MainProcess] =====
=====
[2026-05-18 17:53:00,098: WARNING/MainProcess] =====
[2026-05-18 17:53:00,099: WARNING/MainProcess] Current chunk size: 100
[2026-05-18 17:53:00,099: WARNING/MainProcess] Execution time: 0.0574 sec
[2026-05-18 17:53:00,100: WARNING/MainProcess] Orders processed: 100
[2026-05-18 17:53:00,100: WARNING/MainProcess] =====
=====
[2026-05-18 17:53:00,105: WARNING/MainProcess] =====
[2026-05-18 17:53:00,105: WARNING/MainProcess] Current chunk size: 100
[2026-05-18 17:53:00,106: WARNING/MainProcess] Execution time: 0.0651 sec
[2026-05-18 17:53:00,106: WARNING/MainProcess] Orders processed: 100
[2026-05-18 17:53:00,106: WARNING/MainProcess] =====
```

تم الانتقال إلى Adaptive Chuncking

وهو أسلوب يقوم بتغيير حجم الـ chunk بشكل ديناميكي حسب سرعة التنفيذ.

طريقة العمل:

1. قياس زمن تنفيذ الـ batch الحالية

$$execution_time = time.time() - start_time$$

2. مقارنة الزمن مع Target Time محدد مسبقاً

$$target_time = 0.2$$

3. حساب الحجم الجديد للـ chunk

$$time_ratio = target_time / execution_time$$

4. تكبير أو تصغير حجم الـ chunk حسب الأداء، فإذا كان التنفيذ سريع يتم زيادة حجم الـ chunk أما إذا كان التنفيذ بطيئاً يتم تقليل حجم الـ chunk

تأثير الحل على الأداء:

1. تحسين الأداء
2. تحسين قابلية التوسع
3. تقليل استهلاك الذاكرة
4. تقليل الضغط على قاعدة البيانات

```
[2026-05-18 17:43:00,079: WARNING/MainProcess] =====
====
[2026-05-18 17:43:00,080: WARNING/MainProcess] Current chunk size: 500
[2026-05-18 17:43:00,081: WARNING/MainProcess] Execution time: 0.0360 sec
[2026-05-18 17:43:00,082: WARNING/MainProcess] Orders processed: 500
[2026-05-18 17:43:00,083: WARNING/MainProcess] Next chunk size: 2780
[2026-05-18 17:43:00,083: WARNING/MainProcess] =====
====
[2026-05-18 17:43:00,104: WARNING/MainProcess] =====
====
[2026-05-18 17:43:00,105: WARNING/MainProcess] Current chunk size: 2780
[2026-05-18 17:43:00,106: WARNING/MainProcess] Execution time: 0.0208 sec
[2026-05-18 17:43:00,106: WARNING/MainProcess] Orders processed: 473
[2026-05-18 17:43:00,107: WARNING/MainProcess] Next chunk size: 5000
[2026-05-18 17:43:00,107: WARNING/MainProcess] =====
```

تصحيح الأخطاء:

1. تم إضافة Error Handling داخل الـ Celery Task باستخدام

```
try:
    .....
except Exception as exc:
    raise self.retry(exc=exc)
```

في حال حدوث أي Exception غير متوقعة يقوم Celery بإعادة المحاولة تلقائياً بدلاً من فشل المهمة بالكامل.

2. Retry

max_retries = 3

default_retry_delay = 5

أي إعادة المحاولة 3 مرات كل 5 ثواني بين المحاولات

:Idempotency

تشغيل المهمة أكثر من مرة لن يؤدي إلى نتائج خاطئة أو مضاعفة البيانات

DailySummary.objects.update_or_create()

بدل create ، بذلك لن يتم إنشاء duplicate rows عند تشغيل التاسك أكثر من مرة حيث سيتم تحديث نفس السجل الخاص بنفس التاريخ وهكذا يمنع التكرار وتضارب البيانات.

قبل تطبيق الحلول:

```
[2026-05-18 20:14:38,489: INFO/MainProcess] mingle: searching for neighbors
[2026-05-18 20:14:39,550: INFO/MainProcess] mingle: all alone
[2026-05-18 20:14:39,638: INFO/MainProcess] celery@Sara ready.
[2026-05-18 20:15:00,005: INFO/MainProcess] Task store.tasks.run_fixed_sales_job[d26cb10e-dfe0-49e5-be0d-69670591bba0] received
[2026-05-18 20:15:26,408: WARNING/MainProcess] =====
[2026-05-18 20:15:26,408: WARNING/MainProcess] Execution time: 26.3804 sec
[2026-05-18 20:15:26,409: WARNING/MainProcess] Orders processed: 972027
[2026-05-18 20:15:26,409: WARNING/MainProcess] =====
=====
[2026-05-18 20:15:26,423: INFO/MainProcess] Task store.tasks.run_fixed_sales_job[d26cb10e-dfe0-49e5-be0d-69670591bba0] received
```

في الأنظمة الحديثة، خصوصاً أنظمة التجارة الإلكترونية، يزداد عدد الطلبات الواردة إلى النظام بشكل كبير عند وجود عدد كبير من المستخدمين المتصلين في نفس الوقت. ولضمان استقرار النظام وسرعة الاستجابة، يتم استخدام تقنيات توزيع الأحمال (Load Distribution) لتوزيع الطلبات على عدة خوادم بدلاً من الاعتماد على خادم واحد فقط.

يهدف هذا المتطلب إلى محاكاة طريقة عمل أنظمة توزيع الأحمال المستخدمة في الأنظمة الحقيقية، وذلك من خلال توزيع طلبات المستخدمين على أكثر من خادم باستخدام أكثر من استراتيجية توزيع.

المشكلة:

في حال اعتماد النظام على خادم واحد فقط:

- قد يتعرض الخادم للضغط الزائد .
- ترتفع مدة الاستجابة .
- قد يتوقف النظام عند زيادة عدد المستخدمين .
- تنخفض كفاءة الأداء والاستقرار .

لذلك تم تصميم النظام بحيث:

- يتم توزيع الطلبات على عدة خوادم (Servers).
- يتم منع تكديس الطلبات على خادم واحد .
- يتم استغلال موارد النظام بشكل أفضل.

الاستراتيجيات المستخدمة في النظام:

تم تطبيق نموذج Hybrid Load Balancing والذي يجمع بين Round Robin Load Balancing و Smart Load Balancing.

وذلك لمحاكاة الأنظمة الحديثة التي تستخدم أكثر من أسلوب توزيع للحصول على أفضل أداء ممكن.

1- Round Robin Load Balancing :

تعد خوارزمية Round Robin من أبسط وأكثر طرق توزيع الأحمال شيوعاً. تعتمد هذه الطريقة على توزيع الطلبات بشكل دوري ومتسلسل على الخوادم المتوفرة.

تعد هذه الطريقة:

- بسيطة وسهلة التنفيذ .
- تحقق توزيعاً أولياً متوازناً .
- تقلل من احتمالية تكديس الطلبات على خادم واحد .
- مناسبة للأنظمة التي تكون فيها قدرات الخوادم متقاربة.

إلا أن هذه الطريقة لا تأخذ بعين الاعتبار:

- الضغط الحالي على الخادم .
- عدد العمليات الجارية .
- استهلاك الذاكرة أو المعالج .

فقد يتم إرسال طلب جديد إلى خادم مشغول بالفعل، بينما يوجد خادم آخر أقل ضغطاً، ولهذا السبب تم إضافة أسلوب أكثر آخر داخل النظام.

-2 Smart Load Balancing :

يقوم النظام بمحاكاة أسلوب Smart Load Balancing بحيث لا يتم الاعتماد فقط على التوزيع الدوري، وإنما يتم أيضاً تحليل حالة الخوادم واختيار الخادم الأقل ضغطاً.

يعتمد النظام على مفهوم Load Estimation، أي تقدير الحمل الحالي على كل خادم.

آلية العمل:

يقوم النظام بمراقبة مجموعة من المؤشرات لكل خادم، مثل:

- عدد الطلبات الحالية .
- العمليات النشطة .
- زمن الاستجابة المتوقع .
- مستوى الضغط الحالي .

ثم يتم حساب قيمة تقديرية للحمل (Estimated Load) لكل خادم، وبعد ذلك يتم اختيار الخادم الأقل ضغطاً وإرسال الطلب إليه للحصول على أفضل أداء.

-3 Hybrid Load Balancing

تم استخدام النظام الهجين بهدف الاستفادة من مزايا الطريقتين معاً، ومن خلال الدمج بينهما تم الحصول على توزيع متوازن، واستجابة أسرع.

كيفية تطبيق النظام داخل المشروع

تم إنشاء مجموعة من الخوادم الوهمية (Simulated Servers) ، وكل خادم يمتلك:

- معرف خاص .
- حالة تشغيل .
- قيمة حمل حالية .
- عدد الطلبات النشطة .

وعند وصول طلب جديد يتم تحديد الخادم التالي باستخدام Round Robin وفحص حالة الأحمال الحالية، فإذا كان الخادم المحدد مزدحماً يتم تحويل الطلب إلى خادم أقل ضغطاً، ويتم تحديث بيانات الأحمال بعد تنفيذ الطلب.

GET /api/load-balance

HTTP 200 OK

Allow: GET, OPTIONS

Content-Type: application/json

Vary: Accept

```
{
  "message": "Request distributed successfully",
  "round_robin": "Server-2",
  "smart_choice": "Server-1",
  "servers_status": {
    "Server-1": {
      "requests": 3,
      "load": 55
    },
    "Server-2": {
      "requests": 3,
      "load": 93
    },
    "Server-3": {
      "requests": 2,
      "load": 84
    }
  }
}
```

GET /api/load-balance

HTTP 200 OK

Allow: GET, OPTIONS

Content-Type: application/json

Vary: Accept

```
{
  "message": "Request distributed successfully",
  "round_robin": "Server-3",
  "smart_choice": "Server-2",
  "servers_status": {
    "Server-1": {
      "requests": 3,
      "load": 58
    },
    "Server-2": {
      "requests": 3,
      "load": 50
    },
    "Server-3": {
      "requests": 3,
      "load": 53
    }
  }
}
```